

Protocolo de comunicacion

El cliente y el servidor se comunican por TCP usando la clase **Socket**. Sobre TCP se define un protocolo binario propio.

Principios del protocolo:

- Cada mensaje empieza con un opcode de 1 byte (**uint8_t**).
- Los campos numericos multibyte enviados por red se convierten a network byte order con **htons/htonl** y se leen con **ntohs/ntohl**.
- Los strings variables se mandan como **uint16_t length** seguido de **length** bytes.
- Las estructuras fijas se declaran con **#pragma pack(push, 1)** para evitar padding.
- Los vectores no se mandan como **std::vector**; se manda primero una cantidad y luego los elementos.

Como viajan los datos en ejecucion

Ademas del formato binario, el proyecto organiza el viaje de datos con hilos y colas. La idea es que el loop principal del juego no haga operaciones bloqueantes de red directamente. En vez de eso, se usan objetos intermedios:

- **ClientCommand**: representa una accion que el cliente quiere enviar al servidor.
- **Command**: representa una accion ya recibida por el servidor y lista para ser procesada por el game loop.
- **Response**: representa una respuesta/evento que el servidor quiere enviar a uno o mas clientes.
- **EventClient**: representa un mensaje recibido por el cliente y transformado a un evento interno de UI/render.

Las colas son instancias de **Queue<T>**, una cola bloqueante multiproductor/multiconsumidor. Sus operaciones principales son:

Metodo	Comportamiento
push	Inserta y bloquea si la cola esta llena.
pop	Extrae y bloquea si la cola esta vacia.
try_push	Intenta insertar sin bloquear.
try_pop	Intenta extraer sin bloquear.
close	Cierra la cola y despierta hilos bloqueados.

Cuando una cola cerrada se usa, lanza **ClosedQueue**. Esto se usa como mecanismo normal de cierre para cortar hilos de sender/receiver sin dejar procesos bloqueados.

Flujo cliente hacia servidor

En el cliente, el usuario interactua con SDL: teclado, mouse, chat, inventario, ataque, movimiento, etc. El loop principal no escribe en el socket directamente para cada input. En cambio, crea un comando de cliente y lo mete en **cmd_queue**.

Ejemplos:

- Si el jugador se mueve, se crea `MoveCommandClient`.
- Si hace click sobre un enemigo, se crea `AttackCommandClient`.
- Si escribe en el chat, se crea `ChatCommandClient`.
- Si equipa o desequipa, se crea `EquipCommandClient` o `UnequipCommandClient`.

El flujo es:

```
input del usuario
-> Client::handle_events / process_movement_input
-> cmd_queue.push(ClientCommand)
-> ClientSender::run
-> cmd_queue.pop()
-> ClientCommand::execute(ClientProtocol)
-> ClientProtocol::send...
-> Socket::sendall
-> TCP
```

`ClientSender` corre en un hilo propio. Su trabajo es muy puntual:

```
mientras este vivo:
    cmd = cmd_queue.pop()
    cmd->execute(protocol)
```

El polimorfismo evita que el sender tenga que conocer todos los opcodes. El sender solo sabe ejecutar comandos. Cada comando sabe que metodo de `ClientProtocol` llamar.

Por ejemplo:

```
MoveCommandClient::execute
-> ClientProtocol::sendMove(direction)
-> envia opcode MOVE + direction
```

```
AttackCommandClient::execute
-> ClientProtocol::sendAttack(target_id)
-> envia opcode ATTACK + target_id
```

Esta separacion deja al loop principal libre para seguir renderizando y leyendo input, incluso si la red tarda un poco.

Flujo servidor al recibir comandos

Del lado servidor, cada cliente conectado tiene un `ClientHandler`. Ese handler contiene:

- Un **ServerProtocol**, asociado al socket de ese cliente.
- Un **ClientReceiver** del servidor.
- Un **ClientSender** del servidor.
- Una cola de salida propia para ese jugador (**QueueResp**).

El **ClientReceiver** del servidor corre en un hilo propio y lee comandos desde el socket:

TCP

```
-> Socket::recvall
-> ServerProtocol::readCommand(player_id, commands_queue)
-> parsea opcode y payload
-> crea Command del servidor
-> commands_queue.push(Command)
```

ServerProtocol::readCommand es el punto donde los bytes dejan de ser bytes crudos y pasan a ser comandos del dominio. Por ejemplo:

Opcode recibido	Comando creado
LOGIN	LoginCommand
SIGNUP	SignupCommand
MOVE	MoveCommand
ATTACK	AttackCommand
CHAT o COMMAND	ChatCommand
USE_ITEM	UseItemCommand
DROP_ITEM	DropItemCommand
TAKE_ITEM	TakeItemCommand
BUY_ITEM	BuyItemCommand
SELL_ITEM	SellItemCommand
DEPOSIT_ITEM	DepositItemCommand
WITHDRAW_ITEM	WithdrawItemCommand
DEPOSIT_GOLD	DepositGoldCommand
WITHDRAW_GOLD	WithdrawGoldCommand
EQUIP_ITEM	EquipCommand
UNEQUIP_ITEM	CommandUnequip
DISCONNECT	DisconnectCommand

La cola **commands_queue** es compartida entre los receivers de todos los clientes y el **GameLoop**. Eso significa que todos los clientes producen comandos, pero la logica del mundo los consume en un punto

central.

```
ClientReceiver jugador 1 --\  
ClientReceiver jugador 2 ----> QueueCmd commands_queue -> Gameloop  
ClientReceiver jugador 3 --/
```

Esto evita que cada hilo de cliente modifique el mundo directamente. El estado del juego queda concentrado en el **Game loop**, que decide si un movimiento es valido, si un ataque pega, si una compra se puede hacer, etc.

Si el receiver detecta desconexion o error, empuja un **DisconnectCommand** para que el game loop limpie el modelo de mundo.

Procesamiento en el game loop

El **Game loop** consume comandos desde **commands_queue**, actualiza el estado del mundo y genera respuestas. Las respuestas no se envian directamente desde la logica, sino que se colocan en colas de salida.

El flujo interno es:

```
commands_queue.pop()  
-> ejecutar Command sobre Gameloop / World / Player / NPCs  
-> actualizar estado  
-> crear Response  
-> MonitorQueues::queueTheServerResponse(player_id, response)  
    o MonitorQueues::executeBroadcast(snapshot)
```

Hay dos tipos de envio desde el servidor:

- Respuesta dirigida a un jugador: por ejemplo login, inventario, banco, catalogo de comerciante.
- Broadcast: por ejemplo snapshots periodicos del mundo para todos los jugadores conectados.

MonitorQueues mantiene un mapa:

```
player_id -> QueueResp
```

Cuando se conecta un jugador, **ClientHandler** registra una cola para ese jugador con **addQueuePlayer**. Cuando el game loop quiere responderle, busca esa cola y mete un **Response**.

Flujo servidor hacia cliente

Cada cliente conectado tiene un **ClientSender** del servidor. Este hilo consume respuestas desde la cola de salida de ese jugador:

```

QueueResp outgoing_queue
-> ClientSender::run
-> outgoing_queue.pop()
-> Response::execute(ServerProtocol)
-> ServerProtocol::send...
-> Socket::sendall
-> TCP

```

Igual que en el cliente, el sender del servidor no necesita saber el detalle de cada mensaje. Solo obtiene un **Response** y ejecuta su metodo polimorfico. Cada response sabe que metodo del protocolo usar.

Ejemplos:

Response	Metodo de protocolo usado
ResponseLogin	sendLoginResponse
ResponseSignup	sendSignupResponse
ResponseSnapshot	sendSnapshot
ResponseMap	sendMap
ResponseInventoryUpdate	sendInventoryUpdate
ResponseEquipmentUpdate	sendEquipmentUpdate
ResponseChatMsg	sendChatMsg
ResponseActionError	sendActionError
ResponseTraderCatalog	sendTraderCatalog
ResponseBankContent	sendBankContent

Flujo cliente al recibir datos del servidor

Del lado cliente, **ClientReceiver** corre en otro hilo. Su trabajo es leer mensajes del socket, convertirlos en **EventClient** y ponerlos en **events_queue**.

```

TCP
-> Socket::recvall
-> ClientProtocol::receiveMessage(EventClient)
-> parsea opcode y payload
-> events_queue.push(EventClient)

```

ClientProtocol::receiveMessage interpreta el opcode del servidor:

Opcode recibido	Tipo de evento interno
SNAPSHOT	UPDATE_WORLD

Opcode recibido	Tipo de evento interno
MAP_DATA	MAP_DATA
PLAYER_STATS	OWN_STATS
LOGIN_RESPONSE	LOGIN_RESPONSE
CHANGE_MAP	MAP_CHANGE
CHAT_MSG	CHAT_MSG
ACTION_ERROR	ERROR_ACTION
INVENTORY_UPDATE	INVENTORY_UPDATE
EQUIPMENT_UPDATE	EQUIPMENT_UPDATE
TRADER_CATALOG	OPEN_MERCHANT
BANK_CONTENT	OPEN_BANK

Despues, el loop principal del cliente consume `events_queue` sin bloquear:

```
Client::update_state_from_server
-> while events_queue.try_pop(event)
-> actualizar WorldRenderer / HUD / chat / inventario / estado local
```

Ejemplos:

- `UPDATE_WORLD`: actualiza entidades visibles desde el snapshot.
- `MAP_DATA`: carga el mapa recibido.
- `OWN_STATS`: actualiza barras de vida, mana, oro y experiencia.
- `INVENTORY_UPDATE`: redibuja inventario.
- `EQUIPMENT_UPDATE`: redibuja equipo.
- `CHAT_MSG`: agrega mensajes o globos de chat.
- `OPEN_MERCHANT`: muestra catalogo del comerciante.
- `OPEN_BANK`: muestra contenido del banco.
- `DISCONNECTION`: corta el loop del cliente.

La razon de usar `try_pop` en el loop de render es importante: si no hay eventos del servidor, el cliente no se queda esperando. Sigue procesando input, animaciones y render.

Resumen visual del viaje completo

Comando de cliente hacia servidor:

```
Usuario
-> Client
-> cmd_queue
-> ClientSender
```

```
-> ClientProtocol
-> Socket TCP
-> ServerProtocol
-> Server ClientReceiver
-> commands_queue
-> Gameloop
```

Respuesta del servidor hacia cliente:

```
Game loop
-> Response
-> MonitorQueues
-> QueueResp del jugador
-> Server ClientSender
-> ServerProtocol
-> Socket TCP
-> ClientProtocol
-> ClientReceiver
-> events_queue
-> Client
-> WorldRenderer / HUD / Chat
```

La arquitectura queda dividida así:

Parte	Responsabilidad
Client	Lee input, actualiza UI/render, consume eventos.
ClientSender cliente	Saca comandos de cmd_queue y los envía.
ClientReceiver cliente	Recibe mensajes del servidor y produce EventClient .
ClientProtocol	Serializa comandos y deserializa eventos del servidor.
ClientHandler servidor	Agrupar socket, sender, receiver y cola de salida de un jugador.
ClientReceiver servidor	Lee comandos de red y los mete en QueueCmd .
Game loop	Procesa comandos y modifica el mundo.
MonitorQueues	Administra las colas de respuestas por jugador.
ClientSender servidor	Saca Response de la cola del jugador y las envía.
ServerProtocol	Deserializa comandos y serializa respuestas.

Esta organización permite desacoplar red, lógica y render. La red puede bloquearse en sus propios hilos, el game loop mantiene la autoridad del estado del mundo, y el cliente sigue renderizando aunque todavía no hayan llegado nuevos eventos.

Tipos base

Tipo logico	Representacion
Opcode	<code>uint8_t</code>
Booleano de exito	<code>uint8_t</code> , <code>1</code> para true y <code>0</code> para false
String variable	<code>uint16_t length</code> + bytes
ID de entidad	<code>uint32_t</code>
ID de item en algunos comandos	<code>uint8_t</code> o <code>uint16_t</code> , segun mensaje
Posicion	<code>Position</code> , con coordenadas de 32 bits

Opcodes cliente a servidor

Opcode	Nombre	Uso
1	<code>LOGIN</code>	Login con usuario y password.
2	<code>MOVE</code>	Movimiento en una direccion.
3	<code>ATTACK</code>	Ataque a una entidad.
4	<code>CHAT</code>	Mensaje de chat.
5	<code>USE_ITEM</code>	Usar item de inventario.
6	<code>DROP_ITEM</code>	Tirar item.
7	<code>TAKE_ITEM</code>	Levantar item del piso.
8	<code>COMMAND</code>	Comando textual, por ejemplo meditacion.
9	<code>INTERACT</code>	Interactuar con NPC.
10	<code>BUY_ITEM</code>	Comprar item.
11	<code>SELL_ITEM</code>	Vender item.
12	<code>DISCONNECT</code>	Desconexion ordenada.
13	<code>SIGNUP</code>	Registro de usuario/personaje.
15	<code>CHARACTER_CREATE</code>	Creacion de personaje, reservado/definido en opcodes.
16	<code>DEPOSIT_ITEM</code>	Depositar item en banco.
17	<code>WITHDRAW_ITEM</code>	Retirar item del banco.
18	<code>DEPOSIT_GOLD</code>	Depositar oro.
19	<code>WITHDRAW_GOLD</code>	Retirar oro.
20	<code>LIST_ITEMS</code>	Solicitar contenido de NPC/banco/comerciante.
21	<code>EQUIP_ITEM</code>	Equipar item.
22	<code>UNEQUIP_ITEM</code>	Desequipar item.

Opcode	Nombre	Uso
23	RESURRECT	Solicitar resurreccion.

Mensajes cliente a servidor

Login

```
uint8_t opcode = LOGIN
char name[30]
char pass[30]
```

Los strings son de largo fijo, rellenos con `\0` si son mas cortos. El servidor fuerza el ultimo byte a `\0` al leer.

Signup

```
uint8_t opcode = SIGNUP
char user[30]
char password[30]
uint16_t head
uint16_t body
uint8_t race
uint8_t clase
```

`head` y `body` viajan en network byte order. `race` y `clase` ocupan un byte.

Move

```
uint8_t opcode = MOVE
uint8_t direction
```

`direction` corresponde al enum `Direction`.

Attack

```
uint8_t opcode = ATTACK
uint32_t target_id
```

`target_id` viaja en network byte order.

Chat y command

```
uint8_t opcode = CHAT o COMMAND
uint16_t length
char bytes[length]
```

El mismo formato se usa para mensajes de chat y comandos textuales.

Use item y drop item

```
uint8_t opcode = USE_ITEM o DROP_ITEM
uint32_t instance_id
```

En el cliente actual se envia el slot/index como `uint32_t`, aunque el nombre del campo sea `instance_id`.

Take item

```
uint8_t opcode = TAKE_ITEM
```

No tiene payload.

Interact

```
uint8_t opcode = INTERACT
uint32_t npc_id
uint8_t action
```

`action` distingue la accion sobre el NPC. En el servidor, si `action == 1`, se interpreta como resurreccion con NPC; en otro caso se genera un comando de interaccion general.

Buy item y sell item

```
uint8_t opcode = BUY_ITEM o SELL_ITEM
uint32_t npc_id
uint8_t item_id
uint16_t quantity
```

`npc_id` y `quantity` viajan en network byte order. En la implementacion actual del servidor la cantidad se lee pero no se usa para construir el comando de compra/venta.

Banco: depositar/retirar item

```
uint8_t opcode = DEPOSIT_ITEM o WITHDRAW_ITEM
uint32_t npc_id
uint16_t item_id
```

El cliente arma este mensaje con helper comun para `npc_id + item_id`.

Banco: depositar/retirar oro

```
uint8_t opcode = DEPOSIT_GOLD o WITHDRAW_GOLD
uint32_t npc_id
uint32_t amount
```

List items

```
uint8_t opcode = LIST_ITEMS
uint32_t npc_id
```

Equip item y unequip item

```
uint8_t opcode = EQUIP_ITEM o UNEQUIP_ITEM
uint32_t item_id
```

Disconnect y resurrect

```
uint8_t opcode = DISCONNECT
```

```
uint8_t opcode = RESURRECT
```

No tienen payload.

Opcodes servidor a cliente

Opcode	Nombre	Uso
50	LOGIN_RESPONSE	Resultado de login.
51	SNAPSHOT	Estado periodico del mundo.
52	CHANGE_MAP	Cambio de mapa.

Opcode	Nombre	Uso
53	PLAYER_STATS	Estadísticas propias del jugador.
54	INVENTORY_UPDATE	Actualización de inventario.
55	CHAT_MSG	Mensaje de chat recibido.
56	ACTION_ERROR	Error de acción.
57	SIGNUP_RESPONSE	Resultado del registro.
59	CHARACTER_CREATE_RESPONSE	Resultado de creación de personaje, reservado/definido en opcodes.
61	MAP_DATA	Datos completos del mapa.
62	TRADER_CATALOG	Catálogo de comerciante.
63	BANK_CONTENT	Contenido del banco.
64	EQUIPMENT_UPDATE	Actualización de equipo.

Mensajes servidor a cliente

Respuesta simple

Varias respuestas usan este formato:

```
uint8_t opcode
uint8_t success
uint16_t length
char message[length]
```

success vale **1** si la operación fue exitosa y **0** si falló.

SIGNUP_RESPONSE usa solamente este formato. **LOGIN_RESPONSE** lo extiende agregando el ID del jugador.

Login response

```
uint8_t opcode = LOGIN_RESPONSE
uint8_t success
uint16_t length
char message[length]
uint32_t player_id
```

Chat message y action error

```
uint8_t opcode = CHAT_MSG o ACTION_ERROR
uint16_t length
char message[length]
```

Change map

```
uint8_t opcode = CHANGE_MAP
uint16_t map_id
```

Player stats

```
uint8_t opcode = PLAYER_STATS
uint32_t hp
uint32_t max_hp
uint32_t mana
uint32_t max_mana
uint32_t safe_gold
uint32_t excess_gold
uint32_t exp
uint32_t exp_next_level
uint8_t level
```

Todos los **uint32_t** viajan en network byte order.

Inventory update y equipment update

```
uint8_t opcode = INVENTORY_UPDATE o EQUIPMENT_UPDATE
uint16_t count
MsgSlot slots[count]
```

Cada slot:

```
uint8_t slot_index
uint8_t type_item
uint16_t quantity
```

quantity viaja en network byte order.

Trader catalog

```
uint8_t opcode = TRADER_CATALOG
uint16_t count
repetir count veces:
    uint8_t type_item
    uint32_t purchase_price
    uint32_t selling_price
```

Los precios viajan en network byte order.

Bank content

```
uint8_t opcode = BANK_CONTENT
uint32_t gold
uint16_t count
repetir count veces:
    uint8_t type_item
    uint32_t amount
```

gold y **amount** viajan en network byte order.

Snapshot del mundo

El snapshot es el mensaje periodico principal del servidor. Permite que el cliente reconstruya el estado visible del mundo.

Formato general:

```
uint8_t opcode = SNAPSHOT
uint16_t player_count
PlayerSnapshotData players[player_count]
uint16_t npc_count
NpcSnapshotData npcs[npc_count]
uint16_t item_count
ItemGroundSnapshotData items[item_count]
uint16_t sound_count
SoundEffectSnapshotData sounds[sound_count]
uint16_t visual_count
VisualEffectSnapshotData visuals[visual_count]
```

PlayerSnapshotData

```
char name[30]
uint32_t id
Position position
uint8_t direction
```

```

PlayerStats stats
CharacterTraits ch_traits
uint8_t weapon_id
uint8_t shield_id
uint8_t helmet_id
uint8_t armor_id
uint8_t flags
uint16_t resurrection_time_left_ms

```

Campos convertidos por red:

- `id`
- `position.x`
- `position.y`
- `stats.max_hp`
- `stats.current_hp`
- `stats.current_mana`
- `stats.max_mana`
- `stats.xp`
- `ch_traits.body`
- `ch_traits.head`
- `resurrection_time_left_ms`

`flags` usa bits para estados especiales:

Bit	Constante	Significado
0	<code>PLAYER_FLAG_GHOST</code>	Jugador fantasma/muerto.
1	<code>PLAYER_FLAG_MEDITATING</code>	Jugador meditando.

NpcSnapshotData

```

char name[30]
uint32_t id
uint8_t type_id
Position position
uint8_t direction
uint16_t current_hp
uint16_t max_hp

```

Se convierten `id`, `position.x`, `position.y`, `current_hp` y `max_hp`.

ItemGroundSnapshotData

```

uint8_t item_id
Position position

```

Se convierten las coordenadas de **position**.

SoundEffectSnapshotData

```
uint16_t effect_id
uint32_t pos_x
uint32_t pos_y
```

effect_id corresponde al enum **SoundEffectID**.

VisualEffectSnapshotData

```
uint16_t effect_id
uint32_t recipient_id
uint32_t pos_x
uint32_t pos_y
```

recipient_id = 0 significa broadcast; si no, apunta a un jugador destinatario.

Envio de mapa por red

Aunque los mapas se guardan en TOML en disco, por red se mandan en binario para evitar parseo TOML en el cliente durante el juego.

Formato:

```
uint8_t opcode = MAP_DATA
uint32_t width
uint32_t height
tiles de layer background
tiles de layer details
tiles de layer object
tiles de layer roof
uint16_t citizen_count
CitizenNPCSnapshot citizens[citizen_count]
```

Cada tile ocupa 6 bytes:

```
int32_t sprite_id
uint8_t walkable
uint8_t region
```

El orden de envio es:

```
for layer in [Background, Details, Object, Roof]:
    for y in [0, height):
        for x in [0, width):
            enviar tile(x, y, layer)
```

Los ciudadanos incluidos al final usan:

```
char name[30]
uint32_t id
uint8_t type
Position position
uint8_t direction
```

El cliente recibe esto, reconstruye un objeto **Map** en memoria y guarda los NPCs ciudadanos para renderizarlos/interactuar.

Ciclo completo de comunicacion

Un flujo tipico es:

1. El cliente se conecta por TCP.
2. El cliente envia **LOGIN** o **SIGNUP**.
3. El servidor responde con **LOGIN_RESPONSE** o **SIGNUP_RESPONSE**.
4. Si el login es exitoso, el servidor puede enviar **MAP_DATA** para inicializar el mapa del cliente.
5. El cliente envia comandos de usuario: **MOVE**, **ATTACK**, **CHAT**, **USE_ITEM**, etc.
6. El servidor procesa esos comandos en su cola de comandos.
7. Periodicamente, el servidor envia **SNAPSHOT**, **PLAYER_STATS**, **INVENTORY_UPDATE** y otros eventos.
8. El cliente transforma esos mensajes en eventos internos (**EventClient**) y actualiza render, HUD, chat e inventario.

Consideraciones importantes

Endianness

En la red se usa network byte order para campos de 16 y 32 bits. Esto es importante porque cliente y servidor podrian correr en arquitecturas distintas. Los campos de 1 byte no necesitan conversion.

Packing de structs

Muchas estructuras compartidas usan:

```
#pragma pack(push, 1)
...
#pragma pack(pop)
```

Esto evita bytes de relleno entre campos. Sin esto, `sizeof(struct)` podría variar por alineación y romper el protocolo binario.

Strings

Hay dos formatos de strings:

- Fijos: `char[30]`, usados en login, password y nombres dentro de snapshots.
- Variables: `uint16_t length` + bytes, usados en chat, comandos y mensajes de respuesta.

Vectores

Los `std::vector` nunca se mandan directamente. Siempre se serializan como:

```
uint16_t o uint32_t cantidad  
elementos serializados uno a uno
```

Esto se ve en snapshots, inventario, equipo, catálogo de comerciante, banco y persistencia del mundo.

Diferencia entre archivo y red

El mismo concepto puede tener dos representaciones:

- En disco, el mapa es TOML para poder editarlo con el editor y revisar cambios.
- En red, el mapa es binario para que el cliente lo reciba rápido.

Esa separación es intencional: legibilidad en archivos editables, eficiencia en comunicación.